DALHOUSIE UNIVERSITY

Prediction Models

Nearest neighbours, linear regression, and linear discriminants

Prof. Gabriel Spadon, PhD

Faculty of Computer Science

spadon@dal.ca

1 Three prediction models

Table 1 compares the three methods before the details.

Table 1. Task, assumptions and output compared across kNN, regression and LDA

Method	Task	Assumes	Returns
k nearest neighbours	classification or regression	nothing about the functional form	a prediction only
Multiple linear regression	regression	a linear conditional mean	coefficients and inference
Linear discriminant analysis	classification	Normal classes, one shared covariance	a direction and a rule

Every method trades assumptions against flexibility. kNN assumes nothing about the shape of the function and pays in variance and query cost. The two linear models assume a great deal and gain stability, speed and interpretability. A hyperparameter such as K is a modelling decision, and it is chosen on data the model has not seen.

2 K nearest neighbours

What it is for kNN is the first model of the course. It keeps the training set and answers each new case by looking up the most similar stored cases.[1] It works for classification and for regression.

2.1 The rule

Given a new test case $\mathbf{x}$:

1. Search in the training set for the K most similar cases to the new case.
2. Use the target values of these cases to obtain a prediction:
   - in classification, predict the majority class among the neighbours;
   - in regression, predict the average of the neighbours' target values.

Similarity is measured with a distance. Here it is the Euclidean distance over the space of the p predictors. For two cases $\mathbf{a}$ and $\mathbf{b}$,

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \cdots + (a_p - b_p)^2}.$$

Three properties of this distance matter. It is defined for numeric attributes. It is dominated by whichever attribute has the widest range. It treats every attribute as equally relevant. A categorical predictor first becomes numeric dummy columns (Section 3).

2.2 Worked example: a prediction by hand

Five training cases, two attributes, one nominal target and one numeric target. The query is $\mathbf{x} = (5.0, 3.0)$. Table 2 lists each case with its distance to the query.

- $K = 3$. The three smallest distances are A, B and C. The classification vote is YES, YES, NO, so the prediction is YES. The regression prediction is $(12 + 14 + 22)/3 = 16.0$.

Table 2. Five training cases and their distances to the query point

Case	x_1	x_2	class	y	$d(\mathbf{x}, \mathbf{x}_i)$
A	4.6	3.2	YES	12	$\sqrt{0.16 + 0.04} = 0.447$
B	5.4	3.4	YES	14	$\sqrt{0.16 + 0.16} = 0.566$
C	5.1	2.3	NO	22	$\sqrt{0.01 + 0.49} = 0.707$
D	6.3	3.3	NO	25	$\sqrt{1.69 + 0.09} = 1.334$
E	4.4	1.9	NO	30	$\sqrt{0.36 + 1.21} = 1.253$

- $K = 1$. The prediction is YES and 12.
- $K = 5$. The prediction is NO, by three votes to two, and $(12 + 14 + 22 + 25 + 30)/5 = 20.6$.

One data set and one distance give three different predictions, so K is a modelling decision.

2.3 Scaling the features first

The rule A Euclidean distance adds squared differences, so a variable with a wide numeric range dominates the sum however relevant it is. Standardize every predictor before computing distances.

Worked example A query $\mathbf{x}$ has an income of \$52 000 and an age of 30 years. Table 3 gives two candidate neighbours.

Table 3. Effect of standardization on distances for income and age

	raw units			standardized		
Case	income (\$)	age (yr)	d	income z	age z	d
$\mathbf{x}$	52 000	30	—	−0.20	−0.85	—
P	52 300	61	301.6	−0.18	1.22	2.07
Q	55 000	31	3000.0	0.00	−0.78	0.21

In raw units the nearest neighbour is P, a 61-year-old. The income difference alone contributes 300 to the distance, and the 31-year age gap moves it only to 301.6. After standardizing, the nearest neighbour is Q, a 31-year-old. The data did not change, only the units.

Neither library standardizes for you `class::knn` and `sklearn.neighbors.KNeighborsClassifier` both take the columns exactly as given. Standardize using the *training* mean and standard deviation, then apply the same centre and scale to the test set. Computing them on the combined data leaks test information into the fit.

2.4 Properties, and K as a hyperparameter

- kNN makes no assumptions about the functional form of the mapping.
- kNN is a "**lazy**" learner. "Learning" the model means storing the training set.
- Training is quick. Prediction is more expensive, because every query re-runs the search over all n stored cases.
- K is a **hyperparameter**, a parameter that controls the learning process and is not estimated from the training data the way a coefficient is.

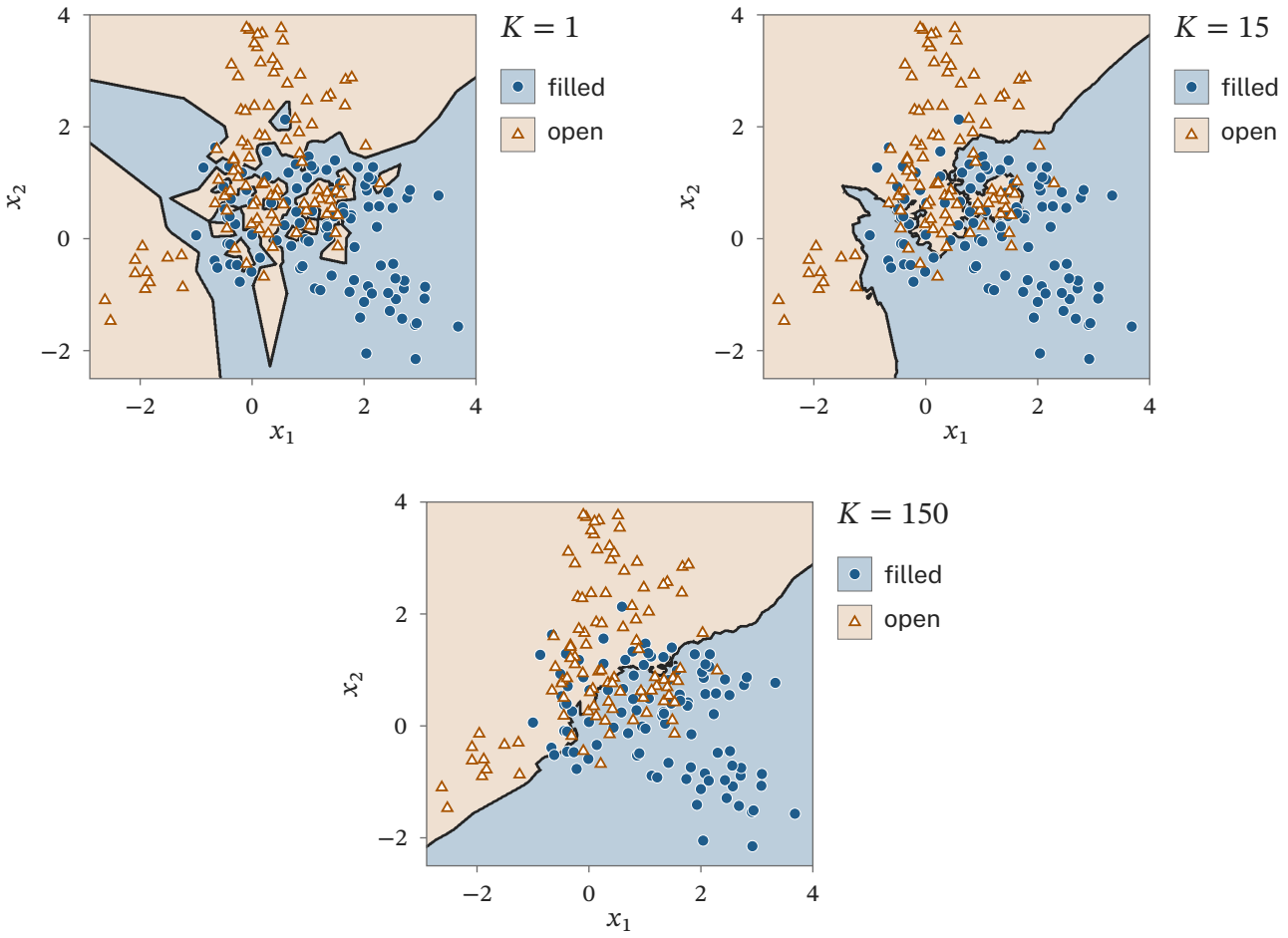


Figure 1. The same 200 training cases classified with $K = 1$, $K = 15$ and $K = 150$. Only K changes. After the mixture example of Hastie, Tibshirani and Friedman [5].

Worked example Figure 1 classifies 200 simulated training cases, described by two numeric predictors x_1 and x_2 and split into two classes of 100, with the same Euclidean distance and code at three values of K. All cases are shown. Blue regions predict filled circles and orange regions predict open triangles.

- $K = 1$, small. Every training case rules its own island of the input space, and the boundary traces the noise in the sample. Small K captures fine-grained patterns but may **overfit**.
- $K = 15$, moderate. Averaging fifteen cases at a time gives one smooth frontier.
- $K = 150$, large. The neighbourhood is most of the training set, the model is no longer local, and the boundary straightens out. Large K is more stable but may **underfit**.

Choosing K Lower-case k and upper-case K name the same hyperparameter. Start the search near $k \approx \sqrt{n}$, where n is the number of training cases. Here $n = 200$, so $\sqrt{n} \approx 14$. This is a starting scale, not an upper bound and not proof that $K = 15$ is best. Validation chooses k. **Global estimation** searches for the ideal k for a given data set. **Local estimation** estimates the ideal k separately for each test case.

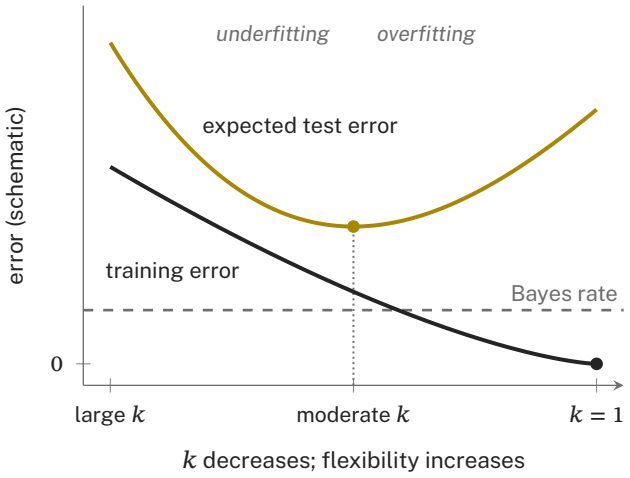


Figure 2. Training and expected test error against flexibility, a qualitative schematic after Hastie, Tibshirani and Friedman [5].

2.5 Overfitting, read off the error curves

The pattern Figure 2 sketches training and expected test error against model flexibility, and Table 4 summarizes it. Neither is a measured benchmark or a guarantee for every data set.

Table 4. Qualitative error patterns as k changes

k regime	Training error	Expected test error
Very small	Can approach zero	May rise through overfitting
Intermediate	Higher	May balance bias and variance
Very large	Often higher	May rise through underfitting

- At $k = 1$, training error is zero when cases include themselves and no identical predictors have conflicting labels. This says nothing about error on new cases.
- As k grows, averaging can reduce variance, but too much averaging adds bias. Expected test error may fall, then rise.
- The **Bayes rate** is a lower bound on expected misclassification error on fresh independent cases from the same distribution. It does not bound training error, and a finite test estimate can fall below it. Its height in the schematic is qualitative.

The rule Never select K on the training curve, because it rewards interpolation rather than generalization. Choose K by validation and assess the chosen model once on untouched test cases.

2.6 The curse of dimensionality

The question kNN needs "nearest" to mean something. Spread the cases evenly over a box and ask how wide a neighbourhood must be, along each axis, to hold just 1% of them.

- With one predictor it spans 1% of the range.
- With three predictors it spans 22% of each range.
- With ten predictors it spans 63% of each range (Figure 3).

The neighbours are no longer near. As p grows the distances between all pairs of points become similar, so the ranking kNN relies on carries less information.

The formula For points uniformly distributed in a unit hypercube of p dimensions, a cubic neighbourhood holding a share r of the cases has side length[5]

$$e_p(r) = r^{1/p}.$$

The numbers above are $r = 0.01$ with $p = 1$, 3 and 10, for instance $0.01^{1/10} = 0.63$.

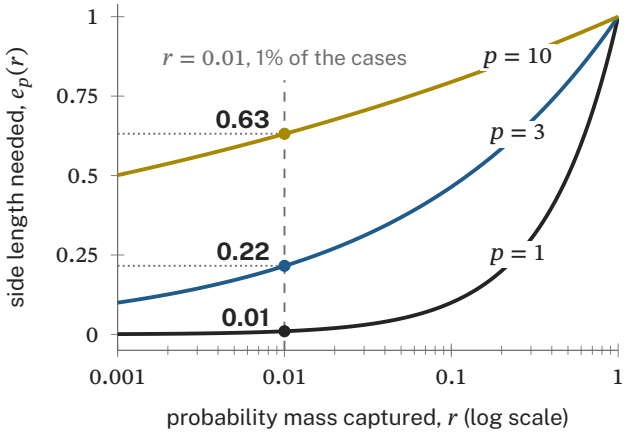


Figure 3. Side length $e_p(r)$ needed to capture probability mass r, for $p = 1$, 3 and 10. At $r = 0.01$ and $p = 10$ the side is already 0.63. After Hastie, Tibshirani and Friedman [5].

The remedies Reduce dimensionality or select features, or weight the distance.

2.7 kNN in R and in Python

The example uses `iris`,[4] 150 flowers, 50 from each of three species (*setosa*, *versicolor* and *virginica*), each measured in centimetres on sepal length, sepal width, petal length and petal width. Both languages ship it. Flower 134 plays the new case and the other 149 are the training set.

R

```r
library(class)
tr <- iris[-134, 1:4]; te <- iris[134, 1:4]
cl <- iris$Species[-134]
set.seed(3141)
knn(tr, te, cl, k = 4)   # ties are broken AT RANDOM

# scale on the training statistics, then apply
# the same transform to the test set
m <- colMeans(tr); s <- apply(tr, 2, sd)
knn(scale(tr, m, s), scale(te, m, s), cl, k = 4)

library(caret)  # choose k by CV on the training rows
train(x = tr, y = cl, method = "knn",
      preProcess = c("center", "scale"),
      trControl = trainControl(method = "cv",
                               number = 5),
      tuneGrid = data.frame(k = c(1, 5, 9, 15)))
```

Python

```python
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import GridSearchCV

X, y = load_iris(return_X_y=True, as_frame=True)
tr, te, cl = X.drop(133), X.iloc[[133]], y.drop(133)
KNeighborsClassifier(n_neighbors=4).fit(tr, cl).predict(te)
# ties: LOWEST CLASS INDEX, deterministically

pipe = make_pipeline(StandardScaler(),
                     KNeighborsClassifier(n_neighbors=4))
pipe.fit(tr, cl).predict(te)  # scaler fitted on tr only

GridSearchCV(pipe,
    {"kneighborsclassifier__n_neighbors": [1, 5, 9, 15]},
    cv=5).fit(tr, cl)          # tuned without the test row
```

Differences to know

1. **Ties.** Row 134 of `iris`, with $k = 4$ and itself excluded, has neighbours 84 and 73 (*versicolor*) and 124 and 127 (*virginica*) at distances 0.332, 0.361, 0.374 and 0.436, an exact 2–2

tie. `class::knn` resolves such a tie *at random*, so the result changes between runs unless a seed is set (over 200 seeds it splits roughly 44/56 between the two classes). `KNeighborsClassifier` returns the class with the *lowest index*, here *versicolor*, every time. Separately, whenever the kth and $(k+1)$th distances tie exactly, which does not happen in this example, R includes *all* of the tied candidates, so the effective neighbourhood can exceed k.

2. **Scaling.** Neither library standardizes by default. In R it is an explicit `scale()` or `caret`'s `preProcess`. In Python it is a `StandardScaler` inside a pipeline.
3. **Weights.** `class::knn` takes an unweighted majority vote. scikit-learn offers `weights="distance"`, which weights each neighbour by the inverse of its distance and therefore gives a different answer for the same k. `caret`'s `kknn` method adds kernel weighting in R.

2.8 When kNN fails

- The features are on incomparable scales and have not been standardized.
- p is large relative to n, because of the curse of dimensionality.
- Many predictors are irrelevant, because they enter the distance with the same weight as the useful ones.
- The training set is so large that scanning it per query is too slow.
- The answer has to be explained, because kNN returns a prediction and nothing else.

3 Handling categorical predictors

What it is for All three methods compute with numbers, so a nominal predictor must be converted before it enters a distance, a regression or a discriminant.

The rule Use **one-hot encoding**.

1. For a variable with C different classes, create $C-1$ "dummy" variables.
2. Fill the new variables with zero if the observation does not belong to the class, or one if it does.

Function `lm()` in R does this automatically when fitting the model.

Worked example Table 5 encodes a river-size variable, `size`, with levels small, medium and large, taken from the algae data used in Section 4.

Table 5. One-hot encoding of the size categorical variable

size	sizemedium	sizesmall
large	0	0
medium	1	0
small	0	1

Three levels give two columns. `large` is the baseline and is recoverable from the two zeros, which is why the third column is dropped. A third column would be redundant, because it always equals the intercept column minus the other two, so `lm` could not estimate a unique weight for each level. The two columns reappear as the `sizemedium` and `sizesmall` coefficients of the algae model in Section 4.

In Python `pd.get_dummies(..., drop_first=True)` follows the same rule. `OneHotEncoder` in scikit-learn keeps all C columns by default, so set `drop="first"` before a regression.

4 Multiple linear regression

What it is for Linear regression makes the opposite bargain to kNN. It assumes the shape of the function, and gains interpretable coefficients, inference and speed. It predicts a numeric target.

4.1 The problem

Multiple linear regression is probably the most used statistical method. It is one of many approaches to the multiple regression problem. The training data is a table with n rows, one per case. The p predictor columns form the matrix $\mathbf{X}$, of size $n \times p$, and the numeric target column is $\mathbf{Y}$, of size $n \times 1$. Row i of $\mathbf{X}$ is case i, and column j is predictor j. The goal is a formula that turns a row of predictor values into a good guess of its target, an approximation of the unknown regression function $f()$.

Every regression model rests on the same relationship,

$$\text{target} = \text{model(predictors)} + \text{error}, \qquad \text{written} \qquad y_i = r(\boldsymbol{\beta}, \mathbf{x}_i) + \epsilon_i,$$

where $r(\boldsymbol{\beta}, \mathbf{x}_i)$ is the model with parameters, or weights, $\boldsymbol{\beta}$, and the ϵ_i are observation errors. Fitting the model means finding the weights that minimize a preference criterion.

That statement contains three separate decisions.

- **The model form** r is a choice. Once it is fixed, the only unknowns are the numbers in $\boldsymbol{\beta}$, and the search for $f()$ becomes an estimation problem.
- **The error term** ϵ_i absorbs everything the model does not explain, meaning measurement noise, omitted predictors and any wrongness in the assumed form. Dropping it would claim the model is exact.
- **The criterion** changes the answer. Squared error, absolute error and likelihood give three different $\hat{\boldsymbol{\beta}}$ from one data set.

A poor fit is usually a problem with the first decision, the model form.

4.2 Least squares

The functional form Multiple linear regression assumes the target is a weighted sum of the predictors,

$$\hat{y} = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p.$$

β_0 is the intercept, the prediction when every predictor is zero, and each other weight says how much the prediction moves per unit of its predictor.

The criterion Each case has a miss, $y_i - \hat{y}_i$, the vertical gap between its true value and the fitted value. Least squares keeps the weights whose **sum of squared errors** is smallest,

$$SSE = \sum_{i=1}^{n} \left(y_i - \hat{y}_i\right)^2.$$

Figure 4 shows the misses for eight cases.

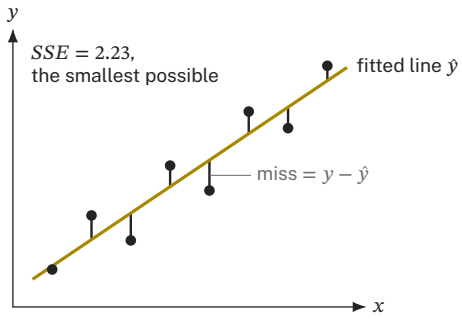


Figure 4. Eight cases, their least-squares line $\hat{y} = 1.546 + 0.637x$, and the misses it leaves. No other line gives a smaller sum of squared misses than 2.23.

Squared error is used rather than absolute error because it treats misses above and below the line alike, it has an exact solution, and it is the maximum-likelihood criterion when the errors are Normal. The cost is that a single outlier can move the whole fit, because a miss twice as large counts four times as much.

How the software solves it Least squares needs no search and no tuning. The weights with the smallest SSE have an exact formula, the **normal equations**, and `lm` solves it in one step. You never compute it by hand. What matters for a data scientist is what the formula needs and what it costs.

Consequences

- **The work.** The cost grows with the number of cases times the square of the number of predictors, so both cases and predictors affect the work. A few hundred rivers and eleven predictors take milliseconds.
- **Distinct columns.** The answer is unique only if no column of $\mathbf{X}$ can be rebuilt from the others. Duplicate a column, or keep all C dummy variables for a factor instead of $C - 1$ while also including an intercept, and there is no unique solution; `lm` reports `NA` for the redundant weight.
- **Near-copies.** Columns that are nearly copies of each other (collinearity) leave the solution defined but make the weights unstable.
- **In practice** R's `lm` uses a QR decomposition and dense scikit-learn `LinearRegression` an SVD-based least-squares routine.

Pros and cons

- Well-known and over-studied topic with many variants of this simple methodology (e.g. Draper and Smith [2]).
- Simple and effective approach when the "linearity" assumption is adequate to the data.
- The form of the model is intuitive, a set of additive effects of each variable towards the prediction.
- Computationally very efficient.
- Too strong assumptions on the shape of the unknown regression function.

kNN, by contrast, assumes nothing about the shape and pays for it in query cost, in sensitivity to scale, and in having nothing to report but a prediction.

4.3 Worked example: fitting the algae model

The data The UCI COIL algae data hold 200 water samples taken from European rivers over about a year. The course bundles the cleaned version distributed with the `DMwR2` package,[11] as three CSV files, the training samples, 140 held-out samples and their published solutions, and R and Python read the same files. The target is `a1`, how frequent one harmful algae species is in a sample, which runs from 0 to 89.8 with a median of 7.0. The eleven predictors are three categories describing the river, `season`, `size` (small, medium, large) and `speed` of the water (low, medium, high), and eight chemical measurements, maximum pH (`mxPH`), minimum oxygen (`mnO2`), chloride (`Cl`), nitrates (`NO3`), ammonium (`NH4`), orthophosphate (`oPO4`), total phosphate (`PO4`) and chlorophyll (`Chla`). Sixteen samples have missing values, and two of them are too incomplete to use.

R

```r
source("mixed_knn_impute.R")
algae <- read.csv("data/algae.csv")
algae <- algae[-c(62, 199), ]
                    # the 2 incomplete samples
predictors <- algae[, 1:11]  # exclude responses
clean.algae <- cbind(mixed_knn_impute(predictors),
                     a1 = algae$a1)
la1 <- lm(a1 ~ ., clean.algae[, 1:12])
la1
```

Python

```python
import pandas as pd
import statsmodels.formula.api as smf
from sklearn.impute import KNNImputer

algae = pd.read_csv("data/algae.csv")  # same file as R
algae = algae.drop(index=[61, 198])
                    # the 2 incomplete samples, 0-based
num = ["mxPH", "mnO2", "Cl", "NO3", "NH4", "oPO4",
       "PO4", "Chla"]
raw_numeric = algae[num].copy()
algae[num] = KNNImputer(n_neighbors=10).fit_transform(
    raw_numeric)
la1 = smf.ols("a1 ~ season + size + speed + "
              + " + ".join(num), algae).fit()
la1.params
```

Three decisions before the fit Two incomplete samples are dropped. The remaining missing values are filled by kNN imputation, the method of Section 2 used as a pre-processing step. The formula `a1 ~ .` takes every remaining column as a predictor.

The imputation helper The R helper, printed in full below, standardizes numeric columns, uses mismatch distance for nominal columns and distance-weights the ten nearest complete rows. Only the eleven predictor columns enter its distances, and response columns never do. Its centres and scales are estimated from the training donors and reused for held-out cases, so a prediction does not depend on which other test cases are in the same batch.

```r
mixed_knn_impute <- function(data, k = 10, donors = NULL) {
  n_target <- nrow(data)
  if (is.null(donors)) {
    pool <- data
    donor_start <- 1L
  } else {
    stopifnot(identical(names(data), names(donors)))
    pool <- rbind(data, donors)
    donor_start <- n_target + 1L
  }

  numeric_cols <- which(vapply(pool, is.numeric, logical(1)))
  nominal_cols <- setdiff(seq_along(pool), numeric_cols)

  distance_data <- pool
  reference <- if (is.null(donors)) data else donors
  for (j in numeric_cols) {
    centre <- mean(reference[[j]], na.rm = TRUE)
    spread <- sd(reference[[j]], na.rm = TRUE)
    if (!is.finite(spread) || spread == 0) spread <- 1
    distance_data[[j]] <- (pool[[j]] - centre) / spread
  }
  distance_data[nominal_cols] <- lapply(
    distance_data[nominal_cols],
    function(x) as.integer(as.factor(x))
  )
  distance_matrix <- as.matrix(distance_data)

  incomplete <- which(!complete.cases(distance_matrix))
  targets <- incomplete[incomplete <= n_target]
  donor_rows <- setdiff(seq.int(donor_start, nrow(pool)), incomplete)
  if (length(donor_rows) < k) {
    stop("Fewer than k complete donor rows are available.")
  }
```

The first part of the helper builds the distance data and checks that enough complete donors exist. The loop below finds, for each incomplete row, its k nearest donors and their weights.

```
  for (i in targets) {
    missing_cols <- which(is.na(distance_matrix[i, ]))
    observed_cols <- setdiff(seq_len(ncol(pool)), missing_cols)
    differences <- sweep(
      distance_matrix[donor_rows, observed_cols, drop = FALSE],
      2,
      distance_matrix[i, observed_cols],
      "-"
    )
    observed_nominal <- intersect(nominal_cols, observed_cols)
    if (length(observed_nominal)) {
      nominal_positions <- match(observed_nominal, observed_cols)
      differences[, nominal_positions] <-
        ifelse(differences[, nominal_positions] != 0, 1,
               differences[, nominal_positions])
    }
    distances <- sqrt(rowSums(differences^2))
    nearest <- order(distances)[seq_len(k)]
    neighbour_rows <- donor_rows[nearest]
    weights <- exp(-distances[nearest])
```

Still inside that loop, each missing value is filled from those neighbours, and the function returns the completed data.

```
    for (j in missing_cols) {
      values <- pool[[j]][neighbour_rows]
      if (j %in% numeric_cols) {
        data[i, j] <- weighted.mean(values, weights)
      } else {
        totals <- tapply(weights, as.character(values), sum)
        data[i, j] <- names(totals)[which.max(totals)]
      }
    }
  }

  data
}
```

Differences to know

- **The rows.** Rows 62 and 199 of the CSV in R are 61 and 198 in pandas, which counts from zero.
- **The formula.** The Python formula interface has no `a1 ~ .`, so the right-hand side is written out. Both reach the same 16 coefficients.
- **The imputer.** The R helper uses $k = 10$, standardizes before measuring distance, distance-weights the neighbours and fills nominal columns. `KNNImputer` defaults to $k = 5$, does not standardize, weights all neighbours equally and accepts numeric columns only, which is why `num` is named explicitly and `n_neighbors` is set by hand. The filled values still differ, and every number quoted below is R's.

The coefficients Table 6 gives the printed coefficients.

`season`, `size` and `speed` are factors, and `lm` has expanded each into dummy variables, one fewer than it has levels. `seasonautumn`, `sizelarge` and `speedhigh` do not appear because they are the baselines against which the others are read (Section 3). A dummy's coefficient compares its level with the

Table 6. Linear regression coefficients for the full algae model

(Intercept)	42.873675	mxPH	−3.545649
seasonspring	3.628157	mnO2	1.024855
seasonsummer	0.707369	Cl	−0.040249
seasonwinter	3.605094	NO3	−1.504920
sizemedium	3.317693	NH4	0.001621
sizesmall	9.749954	oPO4	−0.004792
speedlow	3.818859	PO4	−0.052880
speedmedium	0.229648	Chla	−0.085500

baseline. `sizesmall` $= 9.75$ means a small river is predicted 9.75 frequency units higher than a large one, with the other predictors held fixed.

4.4 Reading `summary(la1)`

What `summary()` reports

- The **distribution of the residuals**, the misses $y_i - \hat{y}_i$, which should be centred on zero, approximately Normal for the tests to hold, and small.
- The **estimate** of each coefficient and its **standard error**, how much the estimate would vary from one sample to another.
- For each coefficient, a t **value**,

$$t = \frac{\text{estimate}}{\text{standard error}} = \frac{\hat{\beta}_i}{s_{\hat{\beta}_i}},$$

  a signal-to-noise ratio, the number of standard errors between the estimate and zero. As a rule of thumb, $|t|$ above about 2 means the weight is clearly away from zero. R flags such coefficients with asterisks in the last column of the table, more of them for a larger $|t|$.
- The **multiple** R^2, the proportion of variance explained. Values near 1 are better.

Where the standard error comes from When the predictors are roughly uncorrelated, the standard error of coefficient j is approximately

$$\mathrm{SE}(\hat{\beta}_j) \approx \frac{\hat{\sigma}}{s_{x_j}\sqrt{n}},$$

where $\hat{\sigma}$ is the residual standard error, the typical miss (17.65 for the full algae model), and s_{x_j} is the standard deviation of x_j. It is the standard error of a mean, $\sigma/\sqrt{n}$, divided by the predictor's spread. Noisier data raise it, and more cases or a more spread-out predictor lower it. When the other predictors can already predict x_j, which is collinearity, the true standard error is larger than this estimate, and for a fully redundant column the coefficient cannot be estimated at all, which is why `lm` reports `NA`.

R^2 compares the model's squared misses, SSE, with the squared misses of always predicting the mean $\bar{y}$, the total sum of squares SST,

$$R^2 = 1 - \frac{SSE}{SST}.$$

Adding a predictor can never raise SSE, so R^2 can only rise as the model grows, and it cannot by itself say how many predictors are too many.

Worked example Table 7 reproduces the coefficient table for the full algae model.

Table 7. Coefficient estimates, standard errors and t-tests for the full model

	Estimate	Std. Error	t value	
(Intercept)	42.873675	24.157169	1.775	.
seasonspring	3.628157	4.140971	0.876	
seasonsummer	0.707369	4.023217	0.176	
seasonwinter	3.605094	3.865707	0.933	
sizemedium	3.317693	3.802592	0.872	
sizesmall	9.749954	4.186566	2.329	*
speedlow	3.818859	4.715964	0.810	
speedmedium	0.229648	3.244008	0.071	
mxPH	−3.545649	2.715984	−1.305	
mnO2	1.024855	0.705522	1.453	
Cl	−0.040249	0.033723	−1.194	
NO3	−1.504920	0.551871	−2.727	**
NH4	0.001621	0.001004	1.615	
oPO4	−0.004792	0.039887	−0.120	
PO4	−0.052880	0.030760	−1.719	.
Chla	−0.085500	0.080332	−1.064	

The fit lines are residual quantiles −37.625, −11.886, −2.688, 7.032, 62.278; residual standard error 17.65 on 182 degrees of freedom; multiple R^2 0.3725. Read them in this order.

1. **Residuals.** The median sits below zero and the maximum is far further from zero than the minimum. The residuals are right-skewed, so the Normal-error assumption is under strain.
2. **Coefficients.** Only two of the sixteen coefficients have $|t|$ above 2, `sizesmall` and `NO3`. With sixteen weights, one or two can get there by chance, so treat neither as conclusive.
3. **Residual standard error.** 17.65 is in the units of `a1`, a target running from 0 to about 90. A typical miss of that size is not a precise model.
4. **R^2.** The model accounts for 37% of the variance of `a1`, and 63% is unexplained. Because adding a predictor can only raise R^2, this figure cannot say whether all sixteen coefficients earn their place. That question is the argument for simplifying the model.

These are in-sample fit statistics. Prediction on held-out data is checked separately, below.

4.5 Simplifying the model

The rule `step()` removes predictors one at a time, each time dropping the term whose removal lowers the AIC most, and stops when no removal lowers it, as set out by Venables and Ripley.[12]

Python has no `step()`, so the loop is written out. It drops whichever term lowers the AIC most and stops when no drop helps.

R

```r
final.la1 <- step(la1)
summary(final.la1)
## lm(formula = a1 ~ size + mxPH + Cl +
##     NO3 + PO4, data = clean.algae[, 1:12])
## Residual standard error: 17.5 on 191
##     degrees of freedom
## Multiple R-squared: 0.3528
```

Python

```python
terms, final_la1 = ["season", "size", "speed"] + num, la1
while terms:     # drop the term that lowers AIC most
    fits = [(smf.ols("a1 ~ " + (" + ".join(
                 t for t in terms if t != d) or "1"),
                 algae).fit(), d) for d in terms]
    m, d = min(fits, key=lambda md: md[0].aic)
    if m.aic >= final_la1.aic:  # nothing helps, so
        break                   # step() stops here
    final_la1, terms = m, [t for t in terms if t != d]
final_la1.summary()
```

The default search runs backward from `la1` with the AIC penalty 2 per parameter, as `extractAIC()` computes it. Two variants change the search.

```r
extractAIC(la1)                   # -2logL + 2*edf
step(la1, direction = "both")     # a removed term may return
step(la1, k = log(nobs(la1)))     # penalty log(n): the BIC path
```

Differences to know

- statsmodels reports an AIC that differs from R's by an additive constant. The constant shifts every candidate at a given step equally, so it does not change which term is dropped.
- scikit-learn's `SequentialFeatureSelector(direction="backward")` is a different procedure. It ranks candidates by a cross-validated R^2 rather than by AIC and has no information criterion to stop on. At its defaults it keeps half the predictors, so where it stops is set by `n_features_to_select` or by a `tol` on the score.

Worked example Table 8 gives the reduced model.

Table 8. Coefficient estimates, standard errors and t-tests for the reduced model

	Estimate	Std. Error	t value	
(Intercept)	57.19004	20.96298	2.728	**
sizemedium	2.81275	3.40208	0.827	
sizesmall	10.41166	3.82430	2.722	**
mxPH	−3.96116	2.48281	−1.595	
Cl	−0.05216	0.03168	−1.646	
NO3	−0.89423	0.35151	−2.544	*
PO4	−0.05914	0.01118	−5.291	***

Residual quantiles are −28.863, −12.723, −3.745, 8.439, 62.942. Nine of the sixteen coefficients go. Multiple R^2 falls slightly, from 0.3725 to 0.3528, and the residual standard error is essentially unchanged, 17.65 against 17.5. The smaller model fits almost as well with less than half the coefficients, and its AIC is lower, so it is the better one.

The t values in Table 8 were computed *after* the same data chose the model, so they are optimistic. Honest model selection needs a separate validation set or a nested cross-validation.

4.6 What a coefficient claims, and a prediction by hand

The claim Take `NO3` $= -0.89423$ from the reduced model. A one-unit increase in `NO3`, *with every other predictor in the model held fixed*, is associated with a decrease of 0.894 in the expected value of `a1`. The coefficient is a conditional association on the predictor's original measurement scale.

Worked example Substitute every fitted term, then add. For a small river with `mxPH` $= 8.0$, `Cl` $= 40$, `NO3` $= 3.0$ and `PO4` $= 120$:

$$\begin{aligned}\hat{y} &= 57.19004 + 10.41166 - 3.96116(8.0) - 0.05216(40) - 0.89423(3.0) - 0.05914(120) \\ &= 24.04653 \approx 24.0.\end{aligned}$$

Because the river is small, the indicator `sizesmall` contributes $+10.41166$. A river of the baseline size would contribute zero there. The result is a fitted mean response of about 24.0 algae-frequency units. It is a point prediction, not a prediction interval for a new observation.

What the coefficient does not claim

- **It is not causal.** Nothing here says that reducing nitrate would raise the algae count.
- **It is not comparable across predictors.** `NH4` has a coefficient of 0.001621 in the full model. That is small partly because `NH4` is measured on a far wider scale, so it does not make `NH4` the least important variable. Compare standardized coefficients, or do not compare.
- **It is not valid outside the data.** "Holding everything else fixed" has no meaning for a combination of predictor values that never occurs in the sample.

4.7 The assumptions, and how each one fails

Table 9 states each assumption, what breaking it does, and how a violation shows up.

Table 9. Linear regression assumptions and how each one can fail

Assumption	What it says	If it breaks	How you would notice
Linearity	The average target is a weighted sum of the predictors	Estimates are biased; the model is wrong, not merely noisy	Curvature in residuals against fitted values
Independence	One case's miss says nothing about another's	Standard errors and tests can be wrong	Time, space or group patterns in the residuals
Constant variance	Misses have the same spread for every case	Coefficients stay unbiased, but tests and intervals are wrong	A funnel shape in residuals against fitted values
Normal errors	Misses follow a bell curve around zero	t values and intervals lose calibration	A bent residual Q-Q plot

The skewed algae residuals call for a check of the Normal-error assumption. `plot(la1)` in R draws all four diagnostic plots. In Python, a residuals-versus-fitted scatter and `scipy.stats.probplot` assess shape and normality, and leverage and influence need separate plots.

4.8 Worked example: prediction on the held-out samples

R

```r
test.algae <- read.csv("data/test_algae.csv")
algae.sols <- read.csv("data/algae_sols.csv")
clean.test.algae <- mixed_knn_impute(test.algae,
    k = 10, donors = algae[, 1:11])
preds <- predict(final.la1, clean.test.algae)
mean((preds - algae.sols$a1)^2)
## [1] 295.5436
plot(algae.sols$a1, preds,
     ylab = 'Predicted Values', xlab = 'True Values')
abline(0, 1, col = 'red', lty = 2)
```

Python

```python
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error

test_algae = pd.read_csv("data/test_algae.csv")
                        # the 140 held-out samples
algae_sols = pd.read_csv("data/algae_sols.csv")
                        # their published answers
imp = KNNImputer(n_neighbors=10).fit(raw_numeric)
                        # the neighbour pool is
test_algae[num] = imp.transform(test_algae[num])
                        # the TRAINING rows only
preds = final_la1.predict(test_algae)
mean_squared_error(algae_sols["a1"], preds)

plt.scatter(algae_sols["a1"], preds)
plt.axline((0, 0), slope=1, color="red", linestyle="--")
plt.xlabel("True Values"); plt.ylabel("Predicted Values")
```

Differences to know

- **The neighbours.** R passes the neighbour pool explicitly as `donors`. In scikit-learn the pool is whatever the imputer was fitted on, so the imputer is fitted on the training frame and only transforms the test frame. Calling `fit_transform` on the test frame would let test rows be their own neighbours, and neither library warns you.
- **Column matching.** `predict.lm` and a statsmodels formula fit match the new frame by column *name*. `LinearRegression.predict` takes a bare array and accepts mis-ordered columns silently.
- **The plot.** `abline(0, 1)` is `plt.axline((0, 0), slope=1)`.
- **The number.** The Python imputation fills different values, so its MSE differs. 295.5436 is R's number.

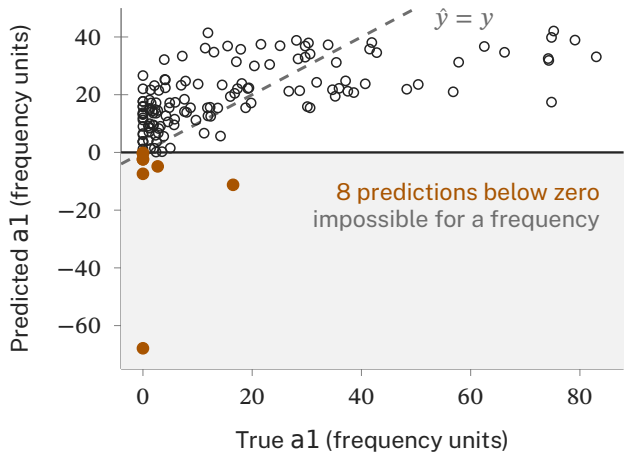


Figure 5. Predicted against true `a1` for the 140 test cases, R pipeline.

Reading the result The test MSE is 295.5436. The scatter in Figure 5 puts true values on the x axis and predictions on the y axis, with the line $y = x$ for reference. Eight of the 140 predictions lie below zero, and there are no negative algae frequencies.

The model cannot know that. Least squares fits a plane over the whole real line, and the constraint $y \geq 0$ was never part of the criterion. Truncating the predictions at zero afterwards would improve this MSE, but it is a repair rather than a model. Modelling the constraint means a different response distribution, a transformation of the target, or a method that respects a floor.

4.9 Linear regression in R and in Python

This parity example uses `iris`,[4] which both languages ship directly (R as a data set, scikit-learn through `load_iris`), so every printed number is identical in the three implementations. The Python listing renames the columns once, and the LDA listings below reuse `iris`.

R

```r
fit <- lm(Petal.Width ~ Petal.Length +
            Sepal.Width + Sepal.Length, data = iris)
coef(fit)
## (Intercept) Petal.Length  Sepal.Width
##    -0.24031      0.52408      0.22283
## Sepal.Length
##     -0.20727
summary(fit)$r.squared
## 0.9379
## residual standard error 0.192 on 146 DF
```

Python

```python
import statsmodels.api as sm
from sklearn.datasets import load_iris
from sklearn.linear_model import LinearRegression

d = load_iris(as_frame=True)  # R's iris, no download
iris = d.frame.set_axis(["sepal_length", "sepal_width",
    "petal_length", "petal_width", "species"], axis=1)
iris["species"] = d.target_names[d.target]
                              # 0, 1, 2 -> setosa, ...
cols = ["petal_length", "sepal_width", "sepal_length"]
X = sm.add_constant(iris[cols])
sm.OLS(iris["petal_width"], X).fit().summary()
# const -0.24031  petal_length 0.52408
# sepal_width 0.22283  sepal_length -0.20727
# R-squared 0.938

LinearRegression().fit(iris[cols],
    iris["petal_width"]).intercept_     # -0.24031
```

Differences to know

1. **The intercept.** `lm` adds one unless the formula says `- 1`. `LinearRegression` adds one unless `fit_intercept=False`. `statsmodels.OLS` adds *nothing*. Without `sm.add_constant` you fit a through-the-origin model whose coefficients and R^2 are not comparable with the others. Its R^2 usually looks *better*, because it is computed against a different baseline.
2. **Factors.** `lm` expands a factor into $C-1$ dummy variables automatically. In Python use `pd.get_dummies(..., drop_first=True)`, or a formula through `statsmodels.formula.api`, which applies the same $C-1$ rule.
3. **Inference.** `summary()` in R and in statsmodels gives standard errors, t values and R^2. `LinearRegression` gives none of them. It is a predictor, not an inferential tool.

4.10 When linear regression fails

- The true relationship is curved and no transformation is applied.
- Predictors are strongly collinear.
- The error variance grows with the fitted value.
- A handful of high-leverage points drive the fit.
- The target is bounded or discrete in a way the model cannot represent, as the negative algae predictions show.

5 Linear discriminant analysis

What it is for LDA keeps the linear form of regression but aims it at the classes instead of at a numeric target. It is a classifier, and it also gives a low-dimensional projection in which the classes separate.

5.1 The idea and the Fisher discriminant

The idea Search for linear combinations of the variables that best separate the objects of the classes. Linear regression looked for a combination that tracks a numeric target. The linear discriminant looks for a combination along which the classes pull apart.

One score per case A set of weights $w_1, \dots, w_p$ turns every case into a single number, its **discriminant score**,

$$z = w_1 X_1 + w_2 X_2 + \cdots + w_p X_p,$$

the same kind of weighted sum a regression uses. Each case becomes one point on a line, and a new case is classified by where its score falls on that line. Figure 6 shows the construction on two classes.

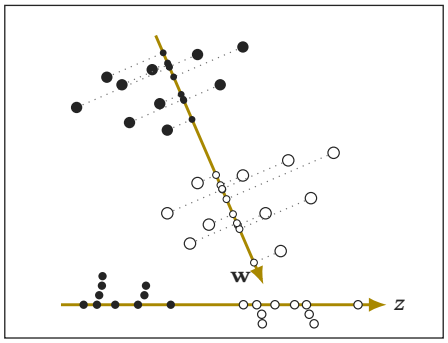


Figure 6. Two classes projected onto the discriminant direction $\mathbf{w}$, and below, the same axis laid flat with one score z per case. On that one axis the classes are separated.

The criterion For two classes, judge a set of weights by the scores it produces. Let $\bar{z}_1$ and $\bar{z}_2$ be the average score in each class and s_1^2 and s_2^2 the variance of the scores within each class. The Fisher separation score[4] is

$$J = \frac{(\bar{z}_1 - \bar{z}_2)^2}{s_z^2}, \qquad s_z^2 = \frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2},$$

where n_i is the number of training cases in class i, so s_z^2 is the within-class variance pooled across the two classes in proportion to their sizes.

- The numerator rewards weights that put the two class averages far apart.
- The denominator charges weights that leave each class widely spread.
- Squaring the gap makes the sign of the weights irrelevant, and multiplying every weight by the same number leaves J unchanged, so only the direction of $\mathbf{w}$ matters.

The solution As with least squares, the weights with the highest J have an exact formula, and `lda` computes them directly from two summaries of the training data: the class means of every predictor, and the **pooled covariance**, the spreads and correlations of the predictors inside each class, averaged over the classes in proportion to their sizes.

- **Separation relative to spread.** The weights push the class means apart *relative to* the within-class spread. A predictor whose class means differ a lot but which is very spread out can count for less than one with a small difference and a tight spread.
- **Correlated predictors.** Because the pooled covariance includes correlations, two predictors that carry the same information are not counted twice. The direction of greatest mean difference is therefore the best direction only in special cases, for instance when every predictor has the same within-class spread and none are correlated.
- **Only the direction matters.** Any nonzero multiple of the weights, including a sign flip, gives the same axis and the same separation of the projected cases, possibly in reversed order.

More than two classes **Canonical discriminant functions** extend the construction. With C classes the method produces at most $C - 1$ discriminant directions, ordered by the share of between-class separation each one carries.

5.2 LDA is not PCA

Both are linear projections that reduce dimensions, and both are used to draw a two-dimensional picture. They optimize different things (Table 10).

Table 10. Principal component analysis compared against linear discriminant analysis

	Principal component analysis	Linear discriminant analysis
Supervision	Unsupervised, never sees the labels	Supervised, uses the labels
Maximizes	Total variance along the projection	Between-class over within-class scatter
Number of axes	Up to p	At most $C - 1$
Answers	"Where does this data set vary?"	"Where do these classes separate?"

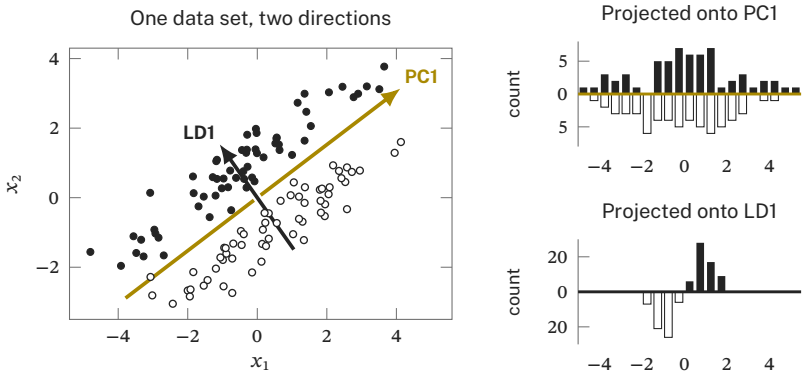


Figure 7. One data set, two directions. Left, the first principal component (gold) and the LDA direction (black), each spanning the projected cases; right, the cases projected onto each, filled class above the line and open class below, in shared bins.

Worked example Figure 7 uses one simulated data set of two elongated, parallel classes whose offset runs across their elongation.

- The first principal component is $(0.795, 0.606)$ and the LDA direction is $(-0.576, 0.817)$. They are **87.9° apart**, nearly perpendicular.

- Projected onto PC1 the classes almost completely overlap, with a standardized mean separation of 0.14 (using the pooled within-class sample standard deviation).
- Projected onto LD1 they are cleanly split, with a separation of 5.05.

PCA reports the direction in which the data vary most, which is correct. It never uses the labels, so it has no reason to keep the classes apart.

Consequence A PCA scatter plot in which the classes do not separate is not evidence that the classes are inseparable. An LDA projection that separates them well is not evidence of low test error, because it was fitted with those labels.

5.3 LDA next to logistic regression

Table 11 compares what each method models, assumes and returns.

Table 11. Linear discriminant analysis compared against logistic regression

	Linear discriminant analysis	Logistic regression
Models	The distribution of $\mathbf{x}$ within each class, then applies Bayes' rule	$P(y \mid \mathbf{x})$ directly
Assumes	Each class Normal with the *same* covariance, one shared spread	Nothing about the distribution of $\mathbf{x}$
Boundary	Linear, as a consequence of the shared covariance	Linear, by construction
Better when	Classes are well separated, or n is small	Assumptions are doubtful, or outliers are present
Also gives	A projection you can plot, and a natural multi-class form	Coefficients on the log-odds scale

Both draw a linear boundary, from opposite directions. LDA is a generative method with real distributional assumptions. When those assumptions hold it is the more efficient of the two, particularly with small samples or well separated classes, where logistic regression's estimates can be unstable.[3] Across the three methods of this class, kNN assumes nothing and returns no direction, linear regression assumes a linear mean, and LDA assumes Normal classes with a shared covariance.

5.4 Worked example: canonical discriminants on iris

scikit-learn stores what `MASS::lda` prints, and on this data set the two agree to every digit shown.

R

```r
library(MASS)
data(iris)
lda(Species ~ ., iris)
## Prior probabilities of groups:
##     setosa versicolor  virginica
##     0.3333     0.3333     0.3333
```

Python

```python
from sklearn.discriminant_analysis import (
    LinearDiscriminantAnalysis as LDA)

X, y = iris.drop(columns="species"), iris["species"]
l = LDA().fit(X, y)
l.classes_     # setosa versicolor virginica
l.priors_      # 0.3333 0.3333 0.3333
l.means_       # the group means, R's "Group means" block
```

Differences to know

- Both default the priors to the class frequencies of the training data, R through `prior` and scikit-learn through `priors`, and both order the classes alphabetically, so the priors and the twelve group means agree.
- R prints the priors, the group means and the coefficients as one block, and `l$counts` gives the class counts (50 50 50). scikit-learn stores each on its own attribute, prints nothing, and leaves the counts to `y.value_counts()`.
- scikit-learn stores the covariance matrix only when `store_covariance=True`. Covariance normalizations differ between the libraries, so compare predictions and discriminant scores rather than stored matrices.

The priors The priors are the class frequencies in the training data, here exactly one third each (R prints its rounding of $1/3$). With two classes they enter the decision rule but not the direction, so changing them slides the boundary along $\mathbf{w}$ without rotating it. With three or more classes `MASS::lda` also weights the between-class scatter by the priors, so the directions move as well, slightly here. With `prior = c(.5, .25, .25)` in R, the LD1 coefficient of `Petal.Width` goes from −2.8105 to −2.7849. With `LDA(priors=[.5, .25, .25])` in Python it moves on scikit-learn's own scale, from −2.8390 to −2.8132.

The group means and coefficients Table 12 gives the group means and the discriminant coefficients.

Table 12. Group means and discriminant coefficients for the iris species

Group means	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
setosa	5.006	3.428	1.462	0.246
versicolor	5.936	2.770	4.260	1.326
virginica	6.588	2.974	5.552	2.026

Coefficients	LD1	LD2
Sepal.Length	0.8294	0.0241
Sepal.Width	1.5345	2.1645
Petal.Length	−2.2012	−0.9319
Petal.Width	−2.8105	2.8392
Proportion of trace	0.9912	0.0088

The group means are the class means LDA pulls apart. The petal columns separate the three species almost on their own, while the sepal columns barely order them. LDA does not have to choose, because it weights all four at once.

Three classes give at most $C - 1 = 2$ discriminants. LD1 carries 99.12% of the between-class separation, so one axis is nearly enough for this data set. Read the coefficient table with three rules.

- **The signs are arbitrary.** Flipping every entry of a discriminant axis, LD1 or LD2, gives the same classifier, mirrored. The sign can differ by platform, R version or LAPACK build, so the LD2 column may print with the opposite sign on another machine and still be correct. Never read a sign as a direction of effect.
- **The magnitudes are not importances.** They are on the scale of the variables, as in a regression.

- **Proportion of trace is a share of *separation*, not variance explained.** It is the LDA analogue of a scree plot, and mistaking it for PCA's explained variance is where the confusion between the two usually starts.

5.5 Worked example: LDA for prediction

R

```r
sp <- sample(1:150, 100)
tr <- iris[sp, ]; ts <- iris[-sp, ]
l <- lda(Species ~ ., tr)
preds <- predict(l, ts)
(mtrx <- table(preds$class, ts$Species))
##              setosa versicolor virginica
##   setosa         16          0         0
##   versicolor      0         18         0
##   virginica       0          1        15
(err <- 1 - sum(diag(mtrx)) / sum(mtrx))
## [1] 0.02
```

Python

```python
from sklearn.model_selection import train_test_split
from sklearn.metrics import (confusion_matrix,
                             accuracy_score)

Xtr, Xts, ytr, yts = train_test_split(X, y,
    train_size=100)               # default 0.75
l = LDA().fit(Xtr, ytr)
preds = l.predict(Xts)
mtrx = confusion_matrix(yts, preds)
                # TRUE down the rows, predicted across
err = 1 - accuracy_score(yts, preds)
```

Reading the result One *versicolor* is called *virginica*, and everything else is correct, so the error rate is $1/50 = 0.02$. The group means predicted this. *setosa* is linearly separable from the other two on petal measurements alone, and the remaining difficulty lies on the *versicolor*–*virginica* boundary.

Two limits of this estimate `sample()` is called without a seed, so the exact table is not reproducible, and a re-run moves the error rate. With 50 test cases the resolution of the estimate is 0.02, one case, so error rates of 0.02 and 0.04 are one flower apart.

Differences to know

- **The split.** `train_test_split` is the counterpart of `sample()`. It takes three quarters of the rows unless `train_size` says otherwise. Neither generator stratifies unless asked (`stratify=y`), so the 50 test cases are not 16 or 17 of each species. Neither is reproducible without `set.seed()` or `random_state=`.
- **The orientation.** R's `table(preds$class, ts$Species)` puts the *predicted* class down the rows. `confusion_matrix` puts the *true* class down the rows. The two tables are transposes, so the cell holding the misread *versicolor* sits on opposite sides of the diagonal.
- **The outputs.** R's `predict` returns `$class`, `$posterior` and `$x` in one object. scikit-learn splits them across `predict`, `predict_proba` and `transform`.

5.6 LDA in R and in Python

R

```r
library(MASS)
l <- lda(Species ~ ., iris)
                  # method = "moment" by default
l$scaling
# LD1: 0.8294 1.5345 -2.2012 -2.8105
l$svd^2 / sum(l$svd^2)
# 0.9912 0.0088  (proportion of trace)
predict(l, iris)$x          # projected scores
predict(l, iris)$posterior  # class probabilities
```

Python

```python
from sklearn.discriminant_analysis import (
    LinearDiscriminantAnalysis as LDA)
l = LDA(solver="svd").fit(X, y)
                    # "svd" is the default solver
l.scalings_
# LD1: 0.8378 1.5501 -2.2236 -2.8390
l.explained_variance_ratio_  # 0.9912 0.0088
l.transform(X)               # projected scores
l.predict_proba(X)           # class probabilities
```

Differences to know

- **Scale.** The two libraries agree exactly on the proportion of trace, but not on the scale of the coefficients. `MASS::lda` divides the pooled covariance by $n - C = 147$ and `solver="svd"` divides it by $n = 150$, so scikit-learn's coefficients are about 1% larger, by the factor $\sqrt{150/147} \approx 1.0102$. A uniform rescaling moves no decision boundary, so the classifier is unaffected. LD2 can also come back with the opposite sign, depending on the platform, which changes nothing either.
- **Solver.** The solver decides what you can extract. scikit-learn's default `solver="svd"` never forms the covariance matrix, so it scales to many predictors, but it cannot do shrinkage and leaves `covariance_` unset unless `store_covariance=True`. `solver="lsqr"` and `solver="eigen"` form it and accept `shrinkage="auto"` (useful when p approaches n), but `"lsqr"` provides no `transform()`, so there is no projection to plot.
- **Estimation.** `MASS::lda` estimates the pooled covariance by moments by default, offers `method="mve"` and `method="t"` for robust alternatives, and always returns `scaling`.

5.7 When LDA fails

- The class covariances differ. The optimal boundary is then quadratic, and the shared-covariance assumption bends it the wrong way. **QDA** drops that assumption at the cost of estimating one covariance per class.
- Outliers are present. They move both the means and the pooled covariance.
- There are too many predictors for the data. The pooled covariance has $n-C$ degrees of freedom, so p must be at most $n-C$, and no predictor may be rebuilt from the others, the same distinct-columns condition least squares needed.

6 Six misconceptions

1. **"kNN has no training, so it has no hyperparameters."** It has K, the distance and the scaling. None of the three is learned from the training data, and all three change the answer.
2. **"A smaller K is more accurate because it uses the closest data."** $K = 1$ can interpolate the training labels yet have poor test error. Training accuracy does not establish generalization.
3. **"R^2 went up, so the model improved."** R^2 cannot go down when a predictor is added. A criterion that charges for size, such as AIC, or an out-of-sample estimate tells you whether the model improved.

4. **"A large $|t|$ means the predictor matters."** It means the estimate is far from zero relative to its standard error, in this model, on this sample, after this selection procedure. With many coefficients, one large $|t|$ alone does not establish usefulness.
5. **"LDA is PCA with labels."** They maximize different quantities and, as Figure 7 shows, can point almost perpendicularly. PCA does not use the labels at all.
6. **"The libraries do the same thing."** They do not. R breaks kNN ties at random and Python by lowest class index; statsmodels adds no intercept; scikit-learn's default LDA solver leaves the covariance matrix unavailable.

7 Summary

kNN

- "Lazy learner": learning consists in storing the training data.
- Prediction is based on the values of the nearest neighbours, and the prediction phase is more computationally demanding.
- Choosing an appropriate K is important; $k \approx \sqrt{n}$ is a starting scale, and validation chooses the value.
- It makes no assumptions about the functional form of the unknown function.

Linear models

- Linear regression finds the linear combination that minimizes the sum of squared errors.
- The linear discriminant finds the linear combination that maximizes a score representing the separation between the classes.
- Both make strong assumptions about the functional form of the unknown function.

The three methods form a ladder of assumptions. kNN assumes only that near cases have similar targets, and pays with variance, query cost and no explanation. Linear regression assumes a linear conditional mean, and gains a closed-form estimator, standard errors and one interpretable coefficient per variable. LDA assumes Normal classes with a shared covariance, and gains a direction that can be plotted and a rule that extends to several classes. Each method is described by what it assumes, what it estimates, and what it returns besides a prediction.

In-class model comparison

The in-class comparison is ungraded. It uses a data set with a numeric target and a nominal one, and fits all three models to it.

- A kNN with K chosen on held-out data and features standardized on the training statistics only.
- A linear model with its coefficients interpreted and its residual diagnostics checked.
- An LDA with its projection plotted.

The models are compared on held-out error metrics, not on accuracy alone, and each comparison names one assumption of each model that the data do not satisfy.

References

[1] T. M. Cover and P. E. Hart. “Nearest neighbor pattern classification”. In: *IEEE Transactions on Information Theory* 13.1 (1967), pp. 21–27.

[2] N. R. Draper and H. Smith. *Applied Regression Analysis*. 2nd ed. Wiley Series in Probability and Mathematical Statistics. Wiley, 1981.

[3] B. Efron. “The efficiency of logistic regression compared to normal discriminant analysis”. In: *Journal of the American Statistical Association* 70.352 (1975), pp. 892–898.

[4] R. A. Fisher. “The use of multiple measurements in taxonomic problems”. In: *Annals of Eugenics* 7.2 (1936), pp. 179–188.

[5] T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning*. 2nd ed. Springer, 2009.

[6] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay. “Scikit-learn: machine learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.

[7] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. Vienna, Austria, 2026.

[8] P. Schoonees. *kNN*. URL: `https://github.com/schoonees/kNN`.

[9] S. Seabold and J. Perktold. “statsmodels: econometric and statistical modeling with Python”. In: *Proceedings of the 9th Python in Science Conference*. 2010, pp. 92–96.

[10] L. Torgo. *Data Mining with R: Learning with Case Studies*. 2nd ed. CRC Press, 2016.

[11] L. Torgo. *DMwR2: Functions and Data for the Second Edition of “Data Mining with R”*. CRAN.

[12] W. N. Venables and B. D. Ripley. *Modern Applied Statistics with S*. 4th ed. Springer, 2002.